John Cheek
Alexander Card
CSC 481
29 September 2025

# Milestone 2 Writeup

This milestone was definitely a lot trickier than the first as not much of our group has prior experience with networking although, most of the challenges came from setting up the multithreading. All aspects of the design were important to consider carefully as they will be expanded upon in the future. I tried to make my personal game project ‘Falling Infinite’ (Figure 1) showcase all these features in an efficient way. In the game, skull shaped ‘coins’ fall endlessly from the top of the game window and the players can collide with a coin to collect it and increase their score. The game can support any number of players and they will need to race to collect coins before others. The timeline is also used to periodically spawn falling entities across the game window, the positions of these entities are then broadcasted by the server and updated client side. The timeline and server side entity handling ensure that the positions of these newly spawned ‘coins’ are consistent across all clients and appear in the same place, regardless of the client’s speed. In addition to networking and the timeline, the main loop of the game demands consistency from the input system and the collision system, which were the two processes we multithreaded as part of task 3.

The timeline is a core aspect of the engine so it was important to consider it was consistent and flexible when designing. When the server runs, the timeline starts counting ‘tics’ and the time delta (calculated from the tics) is used globally by aspects of the engine to handle object movement and the update functions of other engine systems like physics and input. In the future, other systems can use this same time delta to ensure consistent behavior across all engine systems that utilize updates (which is most of them). Because it was built this way, it made asynchronization pretty straight forward as each update function is based on the timeline so the position of all moving objects is kept consistent server side and then broadcast to each client. Our initial design in Milestone 1 enabled an easy transition into the new timeline system as we could just replace every instance where we calculated delta time with the new timeline system. When paused, the timeline system’s own update function sets the time update delta to 0. As a result, every engine system using the timeline (and the time delta) pauses until the engine is

unpaused and the time delta resets to the default tick size of 1.0. This includes all clients, so when one client pauses, the others pause as well. Figure 2 demonstrates a paused server with two networked clients, notice that all entities are paused in the same position across both clients, this is thanks to the asynchronicity provided by the timeline.

Networking was the first major challenge, as many of our group had no prior networking experience. Going into designing, we knew we wanted to store most of the data server side, but primarily the entity positions. That way, the server would always have the ‘correct’ position of all entities. This proved a challenge when implementing the client-server model as we couldn't figure out a good way to handle that while also having the client send their updated positions to the server, so we pivoted to having the clients update their local player positions, then send that to the server. The clients then poll the server and fetch the most recent updates of both objects and other players using the update pattern and the EntityManager. The EntityManager, designed in MS1, allowed us to implement networking easier, as we thought about the fact we would need to add it eventually when designing in MS1. The EntityManager keeps track of the proper positions and data for all entities which is updated whenever the server receives an update from a client. Figure 3 shows the entity manager being used to update the positions of entities within it as part of the client update handler function.

Multithreading was the hardest challenge for this milestone. Based on suggestions in class, our initial design for multithreading targeted putting network updates, input updates, and collision updates each on its own thread. We also understood the boilerplate model and wanted to use that as a framework, and designed our engine in a way that includes a similar design pattern with the shared data and job system. When implementing, I tried to do this all at once which led to some engine breaking errors that provided a major challenge in this milestone. After a while (and help from many debug statements as shown in figure 4) I determined that the input and collisions had been multithreaded properly, but the networking thread was causing issues. This was very difficult to debug as nobody in our group had much experience with networking or multithreading. Since we had collisions and inputs threaded properly, our group decided to do the client-side networking updates outside of a dedicated thread. While this might not have been the optimal design decision, we realized it would be worth it to ensure a consistent and reliable networking system that has no risk of race conditions or deadlocks.

Appendix:

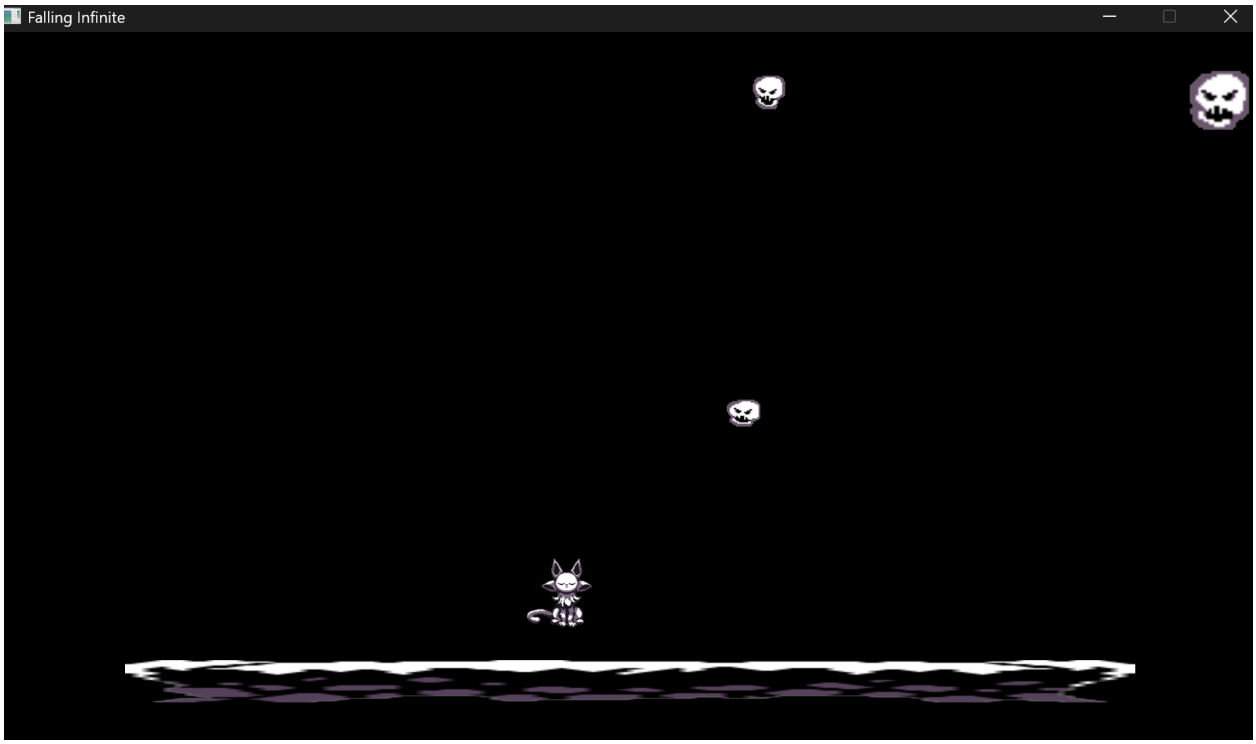


**Figure 1: Falling infinite gameplay screenshot.**

**Figure 2: Falling infinite paused gameplay screenshot with two networked clients .**

```cpp
std::string request_str(static_cast<char*>(request.data()), request.size());
std::istringstream ss(request_str);
std::string cmd, name, xs, ys;
std::getline(ss, cmd, '|');
std::getline(ss, name, '|');
std::getline(ss, xs, '|');
std::getline(ss, ys, '|');

std::string reply_str = "ERR";
if (cmd == "UPDATE" && !name.empty()) {
    try {
        float x = std::stof(xs), y = std::stof(ys);
        std::lock_guard<std::mutex> lock(entityMutex);
        Entity* e = EntityManager::getInstance().findEntityByName(name);
        if (e) { e->x = x; e->y = y; }
        else { EntityManager::getInstance().addEntity(new Entity(name, x, y, 128, 128, nullptr,
        reply_str = "OK";
    }
    catch (...) { reply_str = "ERR"; }
}
```

**Figure 3: Using the entity manager to update entities inside of the server's update handler.**

```
/C/Users/Ultra/OneDrive/Desktop/CSC 481/CSC481-game-engine-team17/build
[Main] New frame
[Main] Entity count: 2
[Job] Collision job started[Job] Input::update() started

[Job] Input::update() complete
[Job] Collision job complete
[Main] New frame
[Main] Entity count: 2
[Job] Input::update() started[Job] Collision job started

[Job] Collision job complete[Job] Input::update() complete

[Main] New frame
[Main] Entity count: 2
[Job] Collision job started[Job] Input::update() started
[Job] Collision job complete
[Job] Input::update() complete

[Main] New frame
[Main] Entity count: 2
[Job] Collision job started[Job] Input::update() started

[Job] Input::update() complete
[Job] Collision job complete
[Main] New frame
[Main] Entity count: 2
[Job] Collision job started[Job] Input::update() started
```

**Figure 4: Debugging statements logging when thread jobs are completed/started and when a new frame starts within the main game loop.**